

JAVA INTERVIEW PREPARATION

CHAPTER 23

LOCKS, ATOMICS, AND COORDINATION

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Locks, Atomics, and Coordination Utilities: Selecting the Right Concurrency Primitive

Senior / Lead Java Developer Reading Chapter

Java provides explicit locks, atomic variables, semaphores, latches, barriers, and other coordination tools because concurrency requirements differ. The goal is not to use the most advanced primitive. It is to choose the smallest mechanism that clearly protects the invariant, handles cancellation, and behaves predictably under contention and failure.

ReentrantLock Versus synchronized

ReentrantLock and synchronized both provide mutual exclusion and memory visibility. ReentrantLock adds operations that built-in monitors do not expose directly: interruptible acquisition, timed acquisition, optional fairness, multiple Condition objects, and explicit lock inspection.

```
private final ReentrantLock lock = new ReentrantLock();

void update() {
    lock.lock();
    try {
        updateState();
    } finally {
        lock.unlock();
    }
}
```

The finally block is mandatory. An exception that skips unlock can permanently block other callers. Use synchronized when its structured syntax expresses the requirement; choose ReentrantLock for a capability the design actually needs.

Timed and Interruptible Acquisition

Explicit locks can avoid waiting forever.

```
if (!lock.tryLock(200, TimeUnit.MILLISECONDS)) {
    throw new BusinessException("Resource is currently unavailable");
}
try {
    updateState();
} finally {
    lock.unlock();
}
```

`lockInterruptibly` allows cancellation while waiting. Timeout and interruption policies must propagate through the call chain; returning a misleading success after failing to acquire the lock corrupts business meaning.

Fair locks broadly favor the longest-waiting thread, but fairness reduces throughput and still does not promise application-level scheduling fairness. Use it only when starvation evidence and latency requirements justify the cost.

Conditions: More Than One Wait Set

A Condition is associated with a Lock and supports await and signal operations. Multiple conditions can represent different predicates under one invariant.

```
private final ReentrantLock lock = new ReentrantLock();
private final Condition notEmpty = lock.newCondition();
private final Condition notFull = lock.newCondition();
```

Like monitor wait, await releases the lock and reacquires it before returning. Always await in a loop.

```
while (queue.isEmpty()) {
    notEmpty.await();
}
```

In application code, a tested BlockingQueue is normally safer than implementing a bounded queue manually. Conditions matter when a custom state machine truly requires them.

ReadWriteLock and Workload Reality

ReentrantReadWriteLock permits concurrent readers while writers are exclusive.

```
read lock: R1 + R2 + R3 may coexist
write lock: W excludes readers and other writers
```

It can help when reads are frequent and sufficiently long, writes are rare, and contention is measurable. For small critical sections, its coordination overhead may exceed any benefit. Readers can also delay writers, depending on policy and traffic.

```
readLock.lock();
try {
    return index.get(key);
} finally {
    readLock.unlock();
}
```

Lock upgrading from read to write is dangerous and can deadlock. Release the read lock and obtain the write lock, then recheck the condition because state may have changed.

StampedLock and Optimistic Reads

StampedLock supports write locks, read locks, and optimistic reads. An optimistic read does not exclude a writer, so every observed state must be validated before use.

```
long stamp = lock.tryOptimisticRead();
double currentX = x;
double currentY = y;
if (!lock.validate(stamp)) {
    stamp = lock.readLock();
    try {
        currentX = x;
        currentY = y;
    } finally {
        lock.unlockRead(stamp);
    }
}
```

StampedLock is not reentrant. Stamps must be paired with the correct unlock operation, and optimistic reads are unsuitable when a temporarily inconsistent read can itself cause failure before validation. It is a specialized optimization, not a default replacement for synchronized.

Compare-and-Set and Atomic Variables

Atomic classes implement operations such as compare-and-set, conceptually:

```
if current value equals expected:
    replace it with update and report success
else:
    report failure
```

```
AtomicInteger remaining = new AtomicInteger(10);

boolean reserveOne() {
    while (true) {
        int current = remaining.get();
        if (current == 0) return false;
        if (remaining.compareAndSet(current, current - 1)) return true;
    }
}
```

The loop retries when another thread wins the race. Lock-free does not mean wait-free: one thread may repeatedly fail, and heavy contention can waste CPU. Use higher-level atomic methods such as `updateAndGet` when they express the operation safely.

Atomicity Has a Boundary

An `AtomicReference` makes reference replacement atomic, not arbitrary mutation of the referenced object.

```
AtomicReference<List<String>> names =
    new AtomicReference<>(List.of());
```

Mutating a shared `ArrayList` stored in the reference remains unsafe. Instead, construct a new immutable value and compare-and-set the entire snapshot.

`AtomicInteger` fields also do not make a multi-field invariant atomic. If debit and credit must move together, independent atomic updates can expose an impossible intermediate state. Use one immutable aggregate in an `AtomicReference`, a lock around the aggregate, or an authoritative transaction.

The ABA Problem

Compare-and-set can observe that a value equals A without knowing it changed from A to B and back to A.

```
Thread 1 reads A
Thread 2 changes A -> B -> A
Thread 1 CAS expects A and succeeds
```

Whether ABA matters depends on what the value represents. `AtomicStampedReference` or versioned immutable state can include a generation. Many application counters do not suffer a meaningful ABA problem, while lock-free linked structures may.

LongAdder and Contended Statistics

`AtomicLong` updates one logical location, which can become a contention hotspot. `LongAdder` spreads updates across cells and combines them for sum.

```
LongAdder requests = new LongAdder();
requests.increment();
long observed = requests.sum();
```

The observed sum need not be an atomic snapshot with concurrent increments. `LongAdder` is excellent for metrics and frequency counts, but wrong for a strict account balance, sequence, or capacity decision.

Semaphore: Protecting Scarce Capacity

A Semaphore controls concurrent permits rather than assigning work to a small thread pool.

```
private final Semaphore permits = new Semaphore(50);

Result callDependency() throws InterruptedException {
    permits.acquire();
    try {
        return client.call();
    } finally {
        permits.release();
    }
}
```

The finally block prevents permit leaks. A timeout may be better than indefinite acquire. Semaphores are especially useful with virtual threads: threads remain cheap while downstream concurrency stays bounded.

A permit limits simultaneous access, not rate per second. Rate limiting needs a time-based algorithm such as token bucket, while a connection pool also owns actual reusable connections.

CountDownLatch, CyclicBarrier, and Phaser

CountDownLatch allows threads to wait until a fixed number of events complete. It is one-shot.

```
CountDownLatch ready = new CountDownLatch(workerCount);
// each worker calls countDown; coordinator calls await
```

CyclicBarrier coordinates a fixed group reaching repeated phases and can run a barrier action. Phaser supports dynamic registration and multiple phases, but its flexibility adds conceptual complexity.

```
CountDownLatch: events count down once -> gate opens permanently
CyclicBarrier: fixed parties meet -> barrier resets
Phaser: dynamic parties advance through numbered phases
```

These utilities coordinate milestones; they do not automatically propagate task failure. A worker that exits before countDown can leave a coordinator waiting forever unless finally blocks, timeouts, and failure channels are designed.

Exchanger and Blocking Queues

Exchanger lets two threads swap values at a rendezvous and fits narrow pipeline designs. BlockingQueue is more common: it combines safe publication, waiting, and often bounded capacity for producers and consumers.

```
BlockingQueue<Job> queue = new ArrayBlockingQueue<>(1_000);
queue.put(job);           // can block when full
Job next = queue.take();  // blocks when empty
```

Choose offer with timeout or rejection when blocking an incoming request is unacceptable. Queue capacity is an operational policy, not merely a constructor detail.

InterruptedException Is Cooperative Cancellation

Interrupting a thread sets its interrupt status and causes certain blocking methods to throw InterruptedException. It does not forcibly terminate arbitrary code.

```
try {
    permits.acquire();
} catch (InterruptedException exception) {
    Thread.currentThread().interrupt();
    throw new OperationCancelledException(exception);
}
```

If a method can declare `InterruptedException`, propagating it is often best. When translating it, restore status unless the method intentionally consumes the cancellation signal. Long CPU loops must check interruption if cancellation is required.

Choosing by Invariant

one compound mutable invariant	synchronized or Lock
independent published flag/reference	volatile
single atomic numeric/reference op	atomic class
high-contention approximate metric	LongAdder
limit simultaneous dependency calls	Semaphore
producer-consumer handoff	BlockingQueue
wait for fixed completion events	CountDownLatch
repeated phase rendezvous	barrier or Phaser

Do not combine primitives casually. A lock plus atomics plus volatile fields around the same state often indicates that the ownership model is unclear.

Production Perspective

Common failures include missing unlock or release in finally, fairness enabled without evidence, `StampedLock` optimistic data used before validation, `LongAdder` used for strict limits, a latch never counted down after an exception, and interrupt status swallowed. Contention can appear as high CPU from CAS retry or as blocked time from locks and semaphores.

Use thread dumps, JFR lock events, queue and permit wait metrics, timeout rates, and CPU profiles. Document what each primitive protects and the permitted acquisition order. The primitive is an implementation detail of the invariant, not the invariant itself.

Interview-Ready Summary

`ReentrantLock` adds timed and interruptible acquisition, fairness options, and `Conditions` beyond `synchronized`. Read-write and stamped locks are workload-specific optimizations. Atomic variables use compare-and-set for operations on a narrow state boundary; they do not make multi-object workflows atomic.

`LongAdder` improves highly contended statistics but does not provide an exact transactional sum. `Semaphore` limits concurrent capacity, `BlockingQueue` coordinates producers and consumers, and latches or barriers coordinate milestones. Every wait needs failure, timeout, interruption, and cleanup semantics.

Revision Notes

- Prefer the simplest primitive that clearly protects the invariant.
- Always unlock `ReentrantLock` in finally.
- `tryLock` and `lockInterruptibly` support bounded or cancellable waiting.
- Fair locks can reduce throughput and are not a default.
- `Condition` awaits must test predicates in a loop.
- `ReadWriteLock` helps only for suitable measured read-heavy workloads.
- Do not upgrade a read lock to a write lock casually.
- `StampedLock` optimistic reads must be validated before use.

- StampedLock is not reentrant.
- Compare-and-set may retry under contention.
- Lock-free does not mean every thread makes bounded progress.
- AtomicReference protects replacement, not mutation of its value.
- Separate atomics do not automatically protect multi-field invariants.
- ABA matters when value history carries meaning.
- LongAdder is designed for contended statistics, not strict balances.
- Semaphore limits simultaneous access, not requests per second.
- Release every permit in finally.
- CountdownLatch is one-shot; barriers coordinate repeated phases.
- BlockingQueue provides safe handoff and can encode bounded capacity.
- Interruption is cooperative and must be propagated or preserved deliberately.